

Group Presentation

P A S E M L 4 0 1 0 1 0
G R O U P 2

Haokun Shi

Rengda Xu

Yuanhang Yang

Yutong Liu

2 2 M A R C H 2 0 2 5

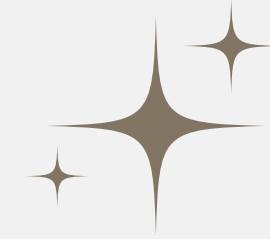
Content Table

- Objective
- Literature Review
- Falstad Simulation
- Octave Codes
- Tinkercad Simulation
- Discussion
- Conclusion
- Reference

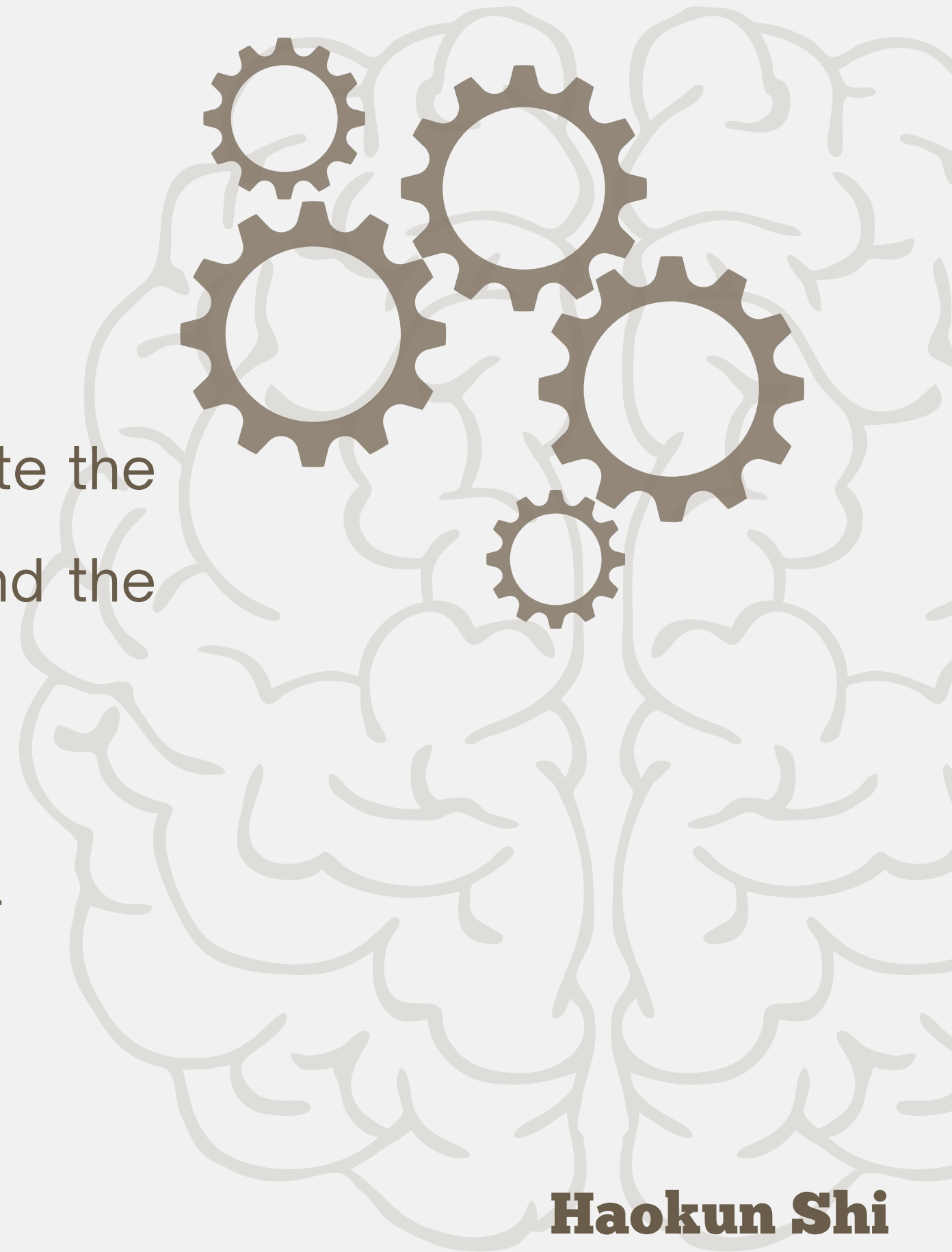


Haokun Shi

Objectives



- Use **Falsetad**, **Tinkercad**, **Octave tools** to compute the values for best performance using proportional and the proportional derivative control for motor angle
- **Challenge** PID control
- **Decrease** the rise/settling time and stead state error



Haokun Shi

Literature Review

- K_p : responsiveness
- K_d : dampen oscillations, reducing settling time.
- K_i : eliminates residual steady-state error by accumulating past errors

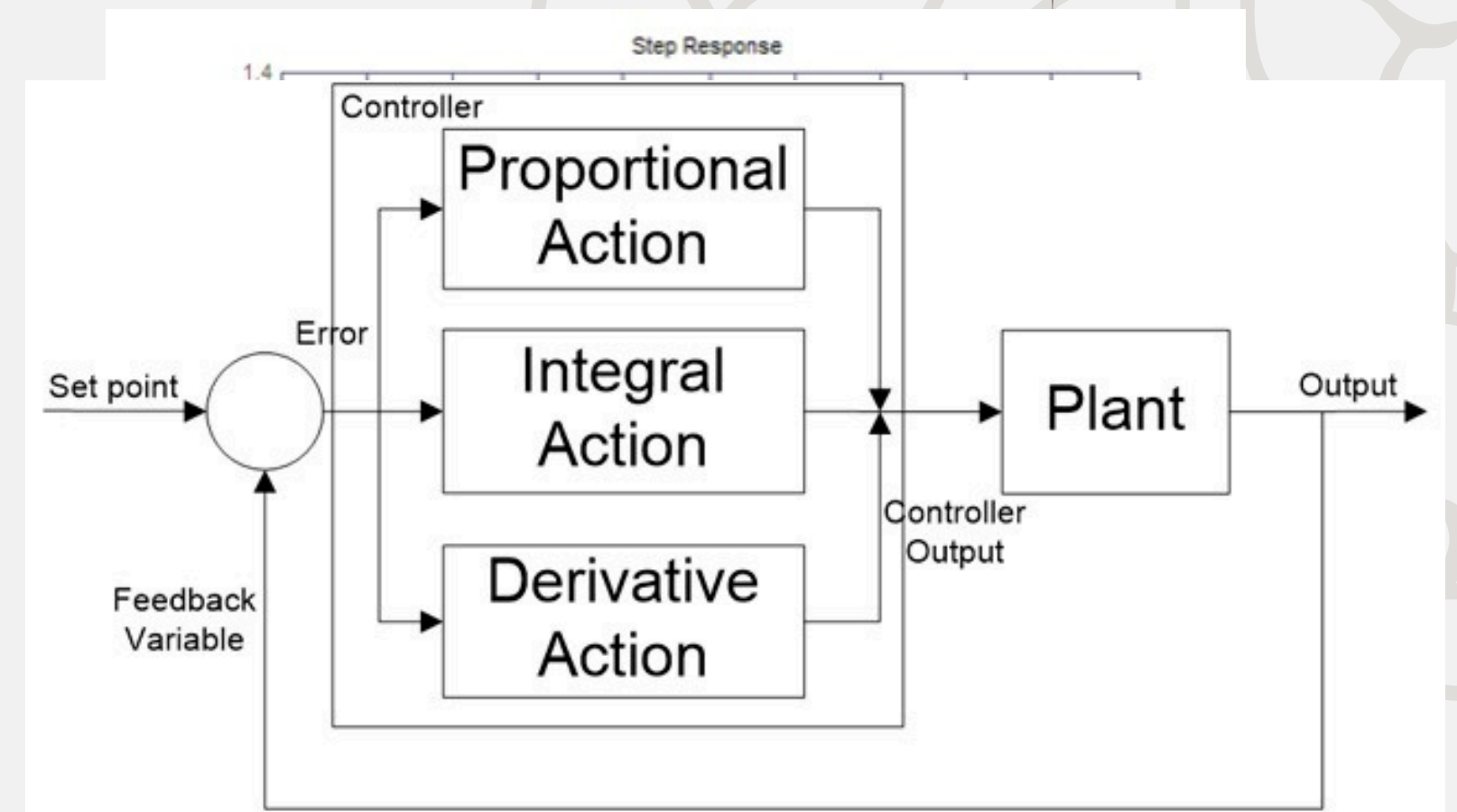
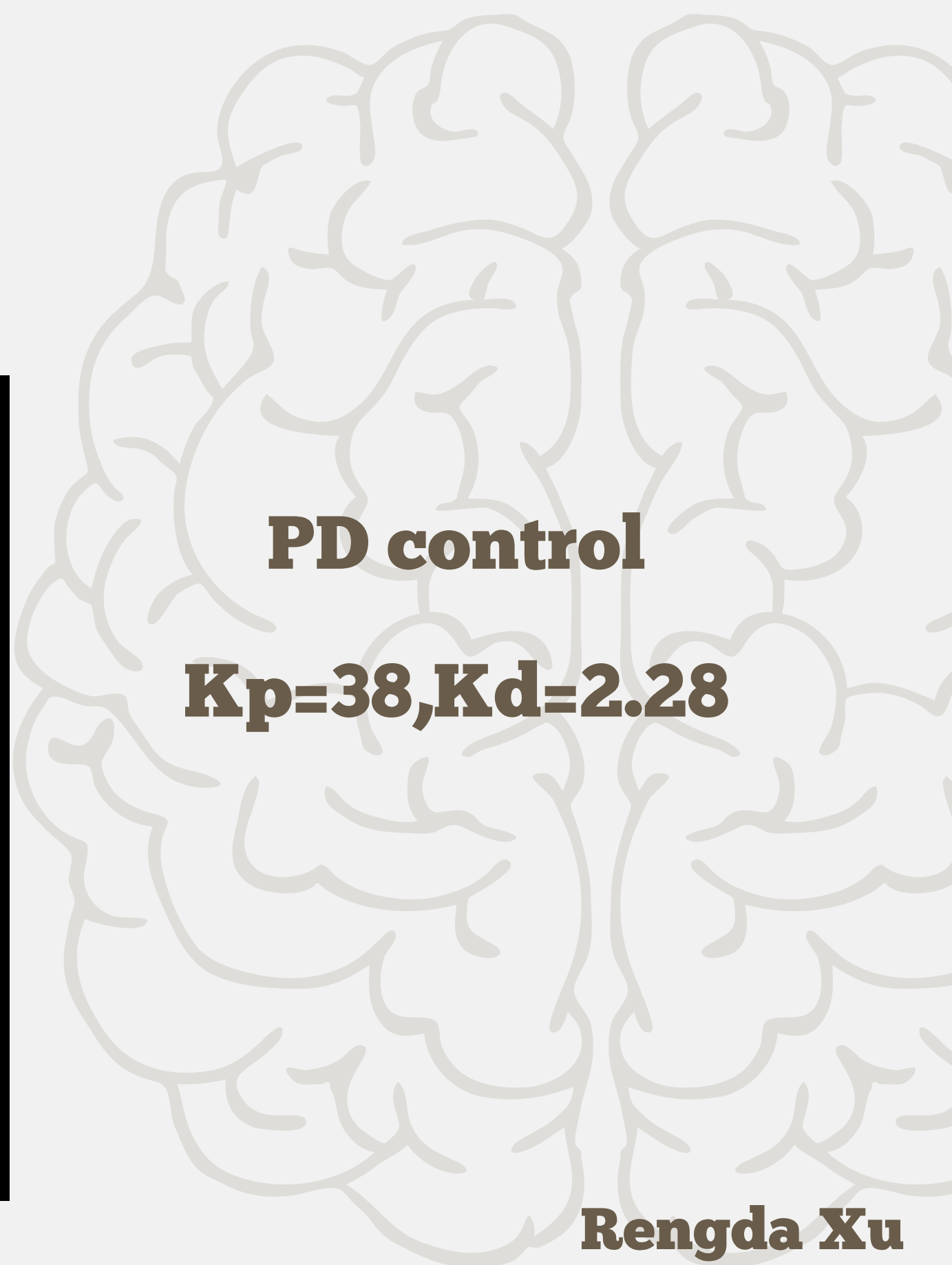


Fig 1. Comparison of P, PD, PID controls (System and Control Theory, n.d.)

Fig 2. Principle of PID controller (Kansagara R, 2018)



Rengda Xu

Result



Orange curve: PD control

Red curve: initial control

Rengda Xu

Analysis

Compared to the red curve, the new curve(orange one) has a smaller rising time, smaller overshoot, shorter settling time, and significantly improved stability compared to the initial PD control.

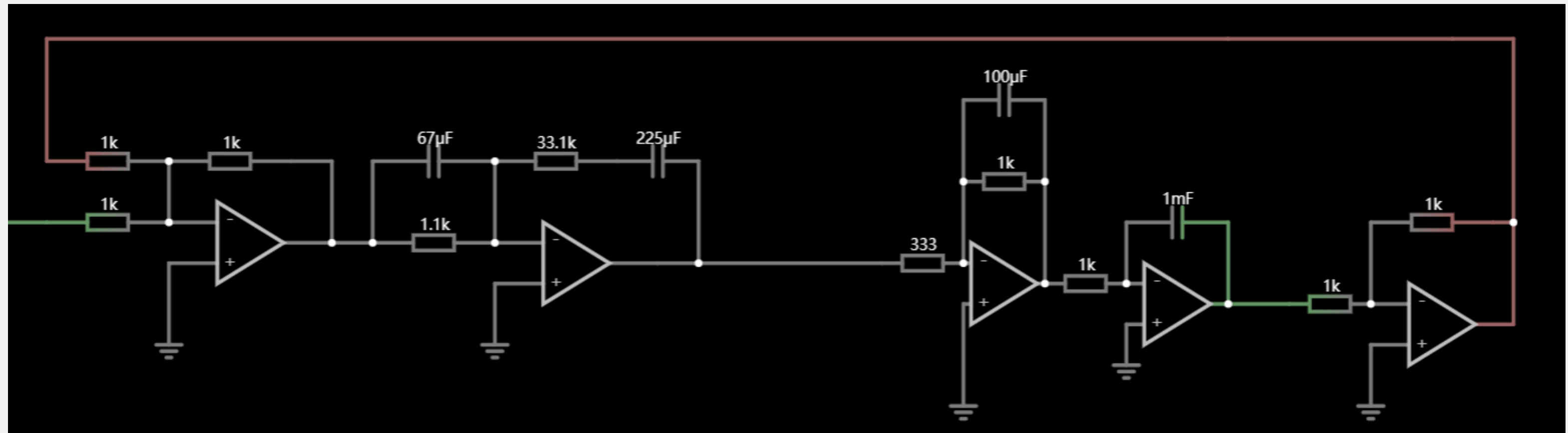
PID control

$$K_p = \frac{R_2}{R_1} + \frac{C_1}{C_2}$$

$$K_i = \frac{1}{C_2 R_1}$$

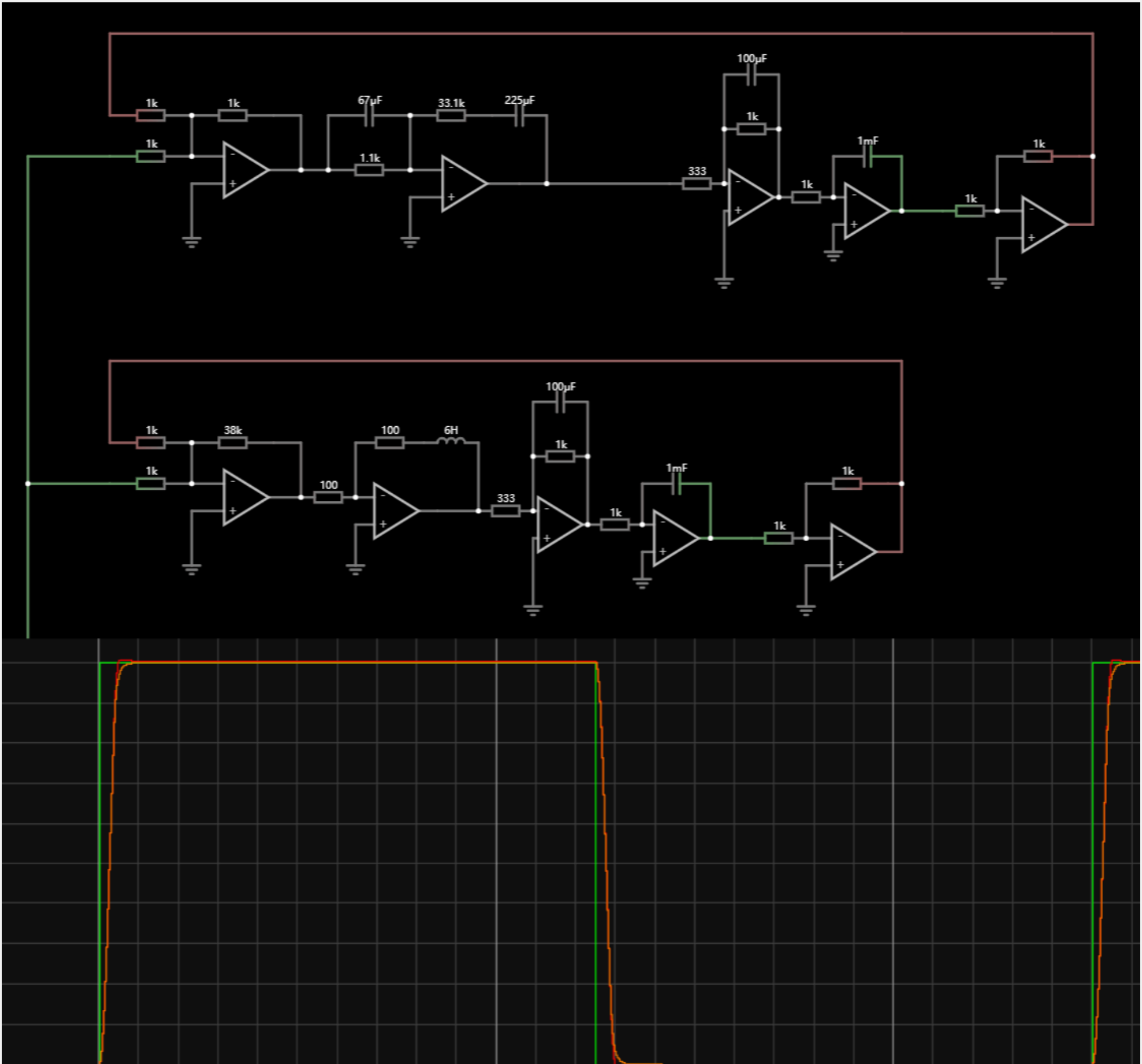
$$K_d = R_2 C_1$$

K_p=30 K_d=2.2 K_i=4



Rengda Xu

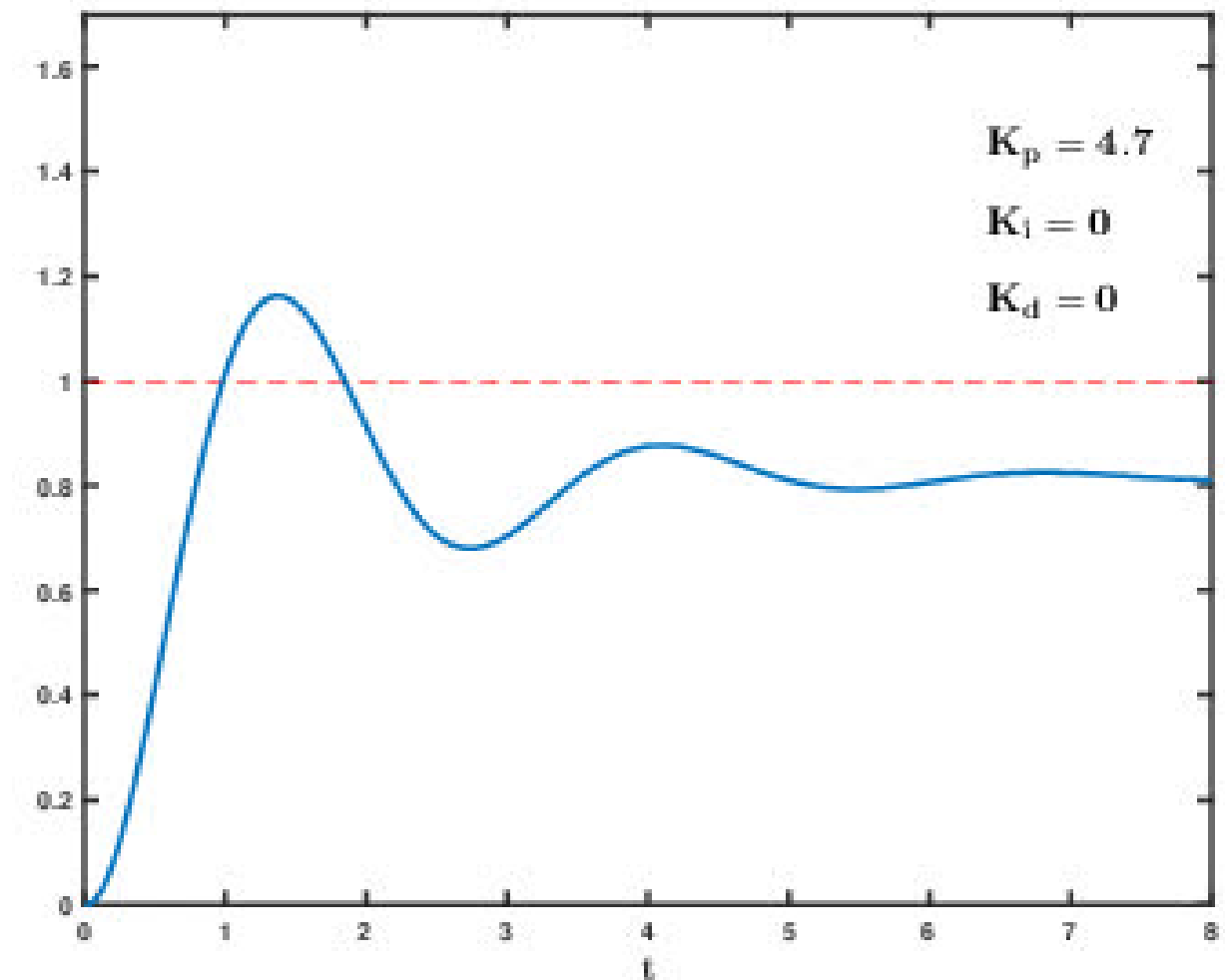
Result



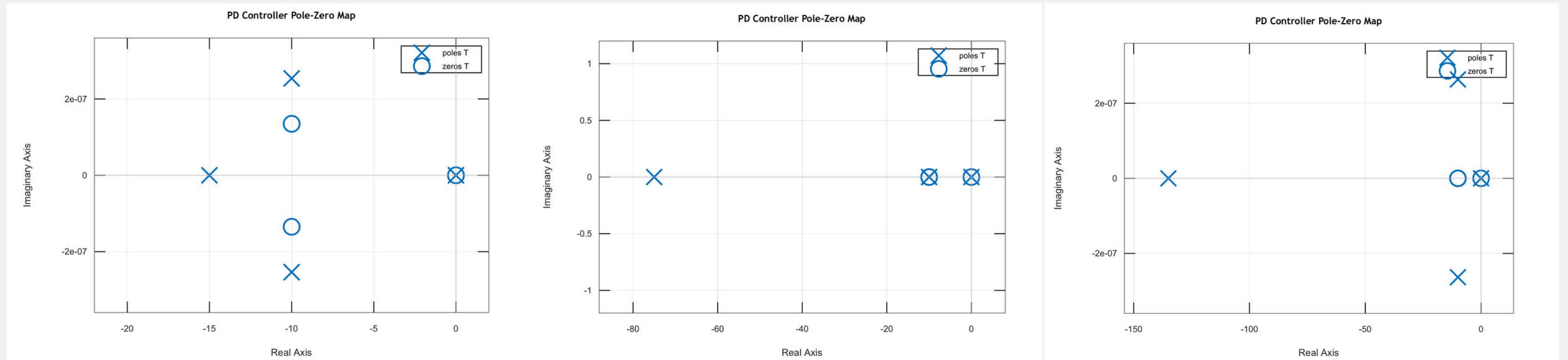
Rengda Xu

Octave

- **K_p increase: Increase responsiveness**
overshoot and oscillations
- **K_i Increase: Reduces steady-state error**
lead to instability and overshoot
- **K_d increase: Improves damping**
reducing overshoot and settling time



Kd and Kp increase drifts poles to the left, meaning more stabilized.



```

pkg load control

s = tf('s'); % Transfer function variable
G = 3 / (s^2 * 0.1 + s); % Second-order system G(s)

Kp = 38; % Reduce proportional gain to improve stability
Kd = 2.2; % Increase derivative gain to add damping and reduce oscillations

% Design PD controller
C_pd = Kp + Kd * s;

% Compute closed-loop transfer function (closed-loop system)
T = C_pd * G / (1 + C_pd * G);

% Compute step response
t = 0:0.01:5; % Simulation time
[y_pd, t_pd] = step(T, t);

% Setpoint
setpoint = 1;
y_ff = setpoint * ones(size(t_pd)); % Output with setpoint

% Manually calculate overshoot and settling time
settling_time = t_pd(find(abs(y_pd - setpoint) < 0.02 * setpoint, 1, 'first')); % Settling time
overshoot = max(y_pd) - setpoint; % Overshoot

% Plot response graph
figure;
plot(t_pd, y_pd, 'b', 'LineWidth', 2); grid on;
hold on;
plot(t_pd, y_ff, 'r--', 'LineWidth', 1.5); % Setpoint
title('PD Response with Enhanced Stability');
xlabel('Time (s)');
ylabel('Response');
legend('PD Response', 'Setpoint', 'Location', 'northeast');
hold off;

% Compute rise time
final_value = y_pd(end);
rise_time_idx = find(y_pd >= 0.9 * final_value, 1, 'first');
rise_time = t_pd(rise_time_idx);

% Output performance metrics
fprintf('PD Parameters: Kp=%.2f, Kd=%.2f\n', Kp, Kd);
fprintf('Settling Time: %.2f s\n', settling_time);
fprintf('Overshoot: %.2f\n', overshoot);
fprintf('Rise Time: %.2f s\n', rise_time);

% Plot Bode diagram
figure;
bode(T);
title('PD Controller Bode Plot');
grid on;

```

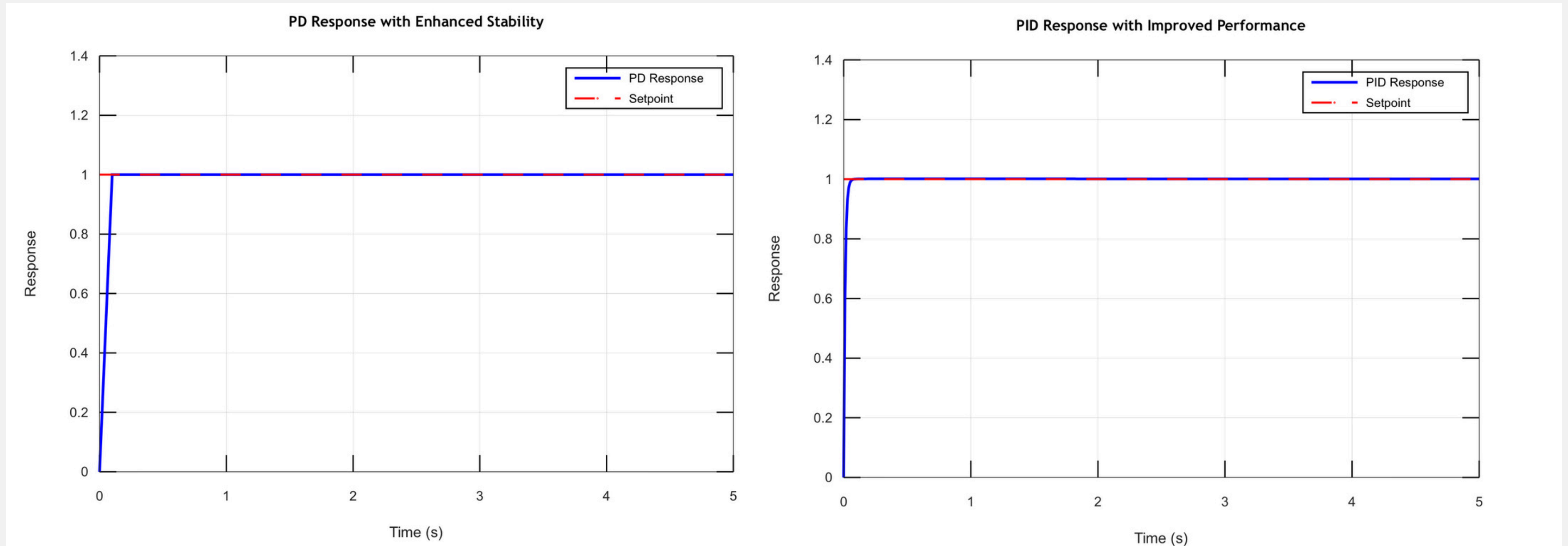
```

56 % Display system gain and phase margin
57 [GM_pd, PM_pd, Wcg_pd, Wcp_pd] = margin(T); % PD gain margin, phase margin
58 % Output gain margin and phase margin information
59 fprintf('PD Controller - Gain Margin: %.2f, Phase Margin: %.2f\n', GM_pd, PM_pd);
60
61 % Pole-zero map
62 figure;
63 pzmap(T);
64 title('PD Controller Pole-Zero Map');
65 grid on;

```

Printed for: yutonl48@uci.edu
 Powered by Octave Online
<http://octave-online.net>

Yutong Liu



It contains poles that are far from the origin, as well as some poles close to the origin.

There are fewer zeros, which **affects system stability and error compensation**.

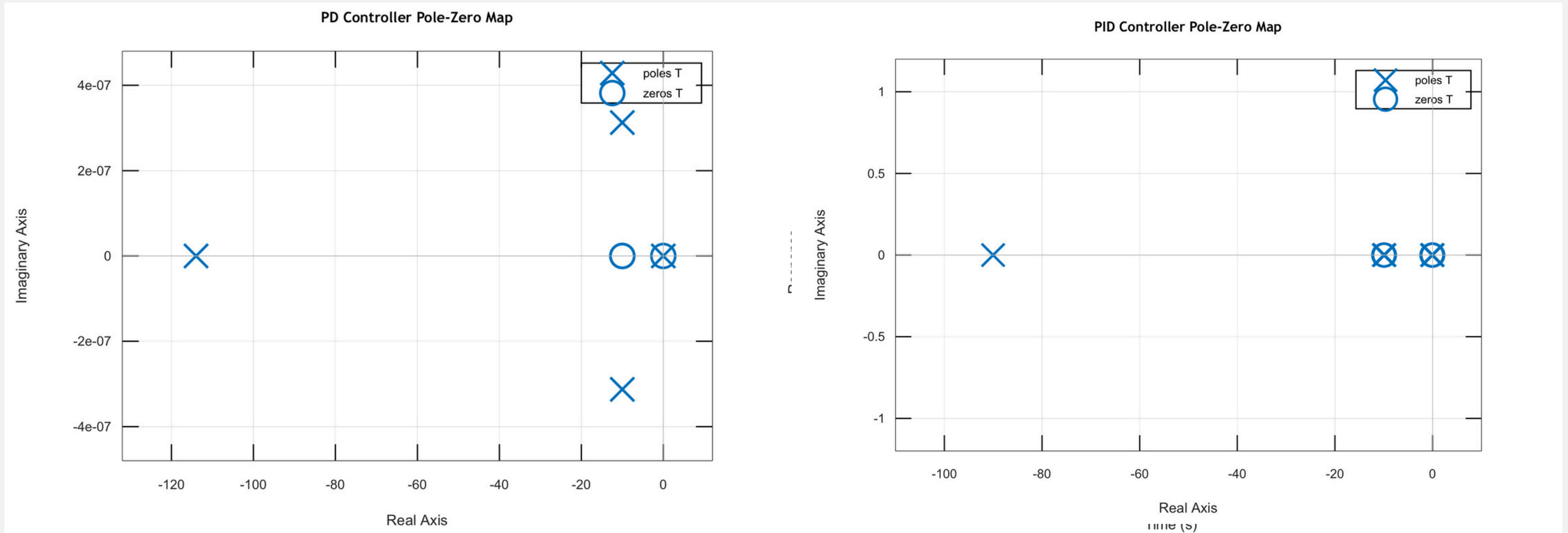
Eliminate steady-state error, with relatively smooth performance.

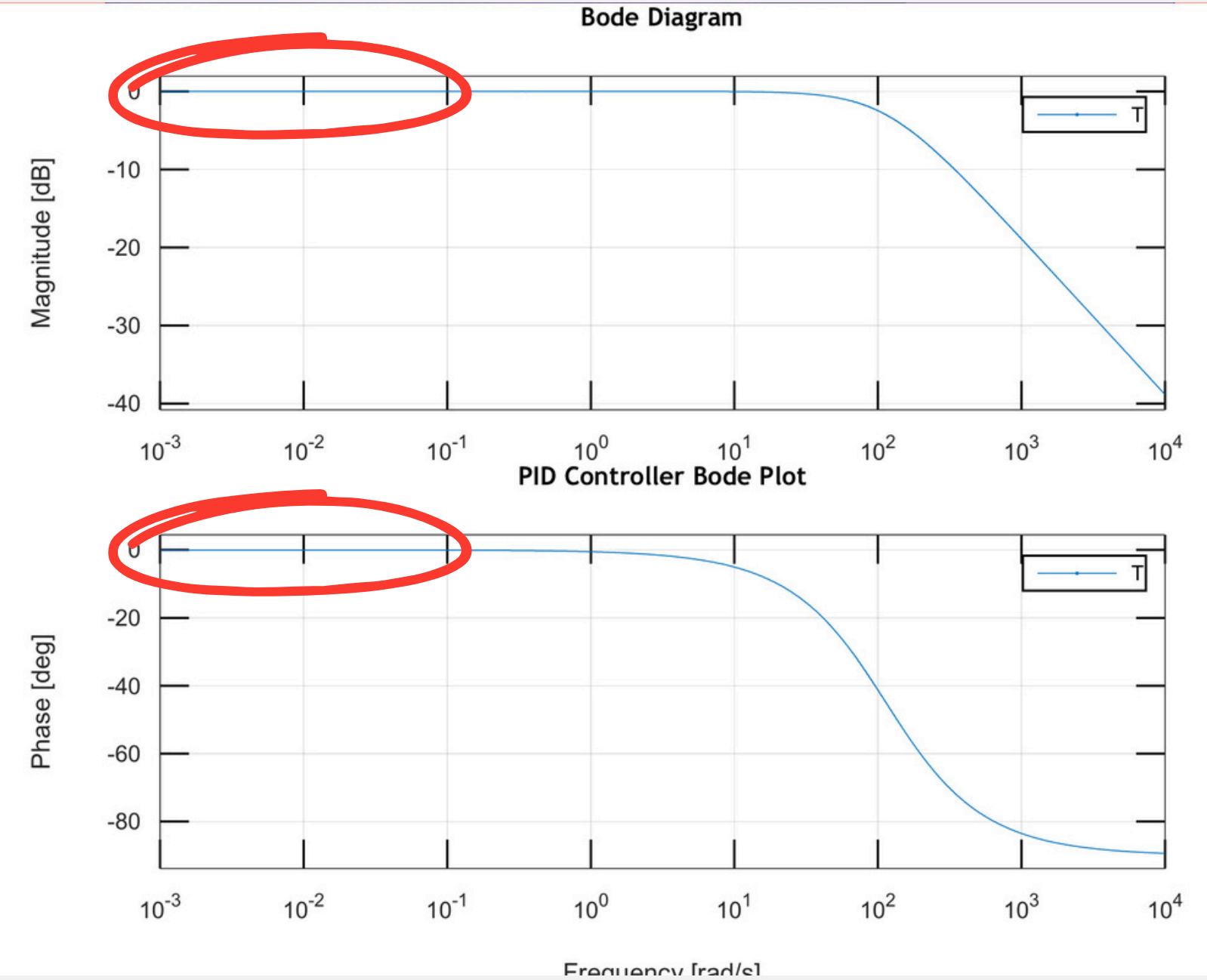
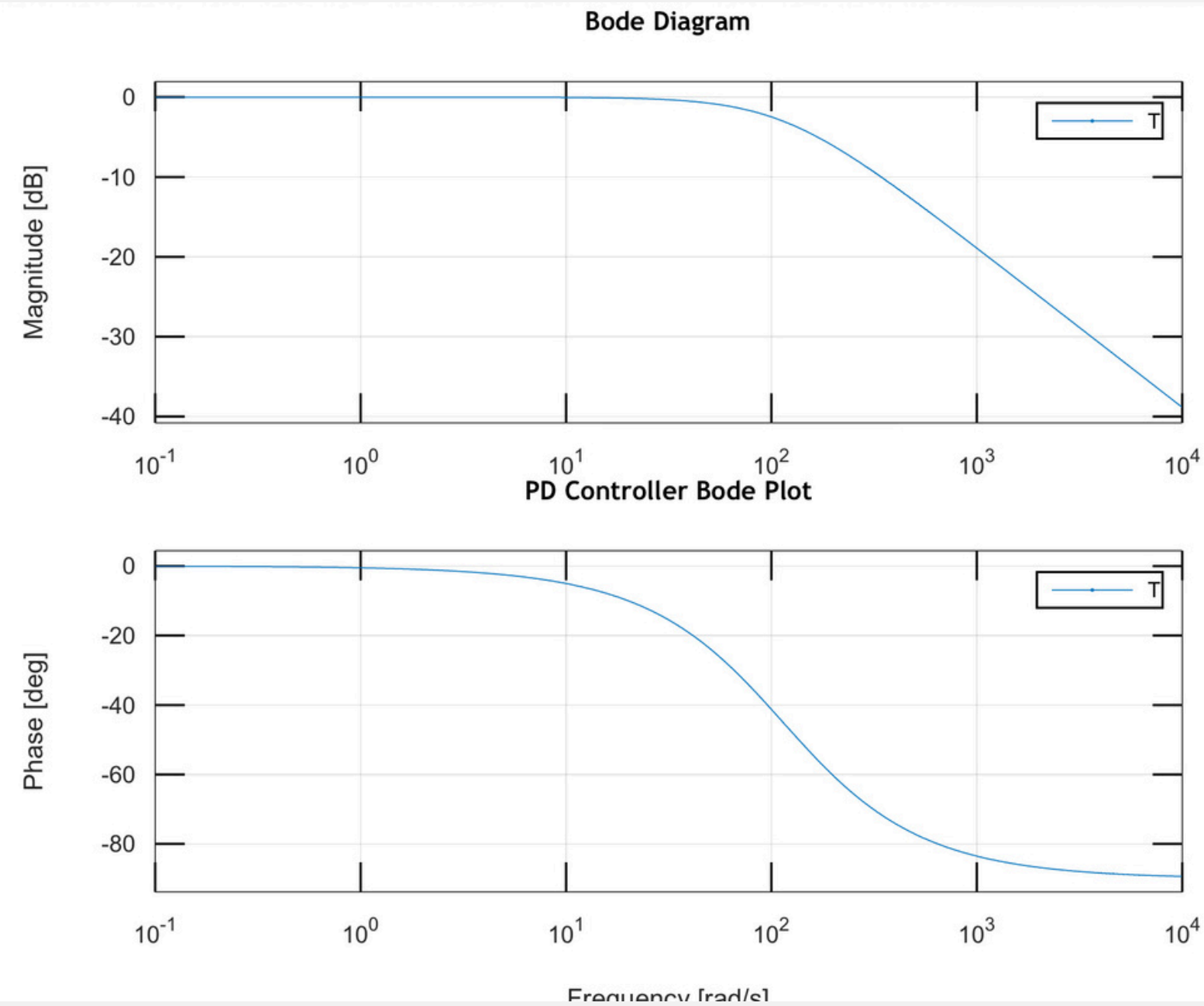
Yutong Liu

There is still a distant pole, and some poles **deviate** from the real axis (which may affect dynamic response).

Additional zeros are used to improve system response speed.

fast response.

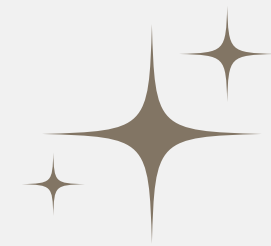




The effects of K_i is not obvious because of relatively small value.

It can offer response in low frequency area.

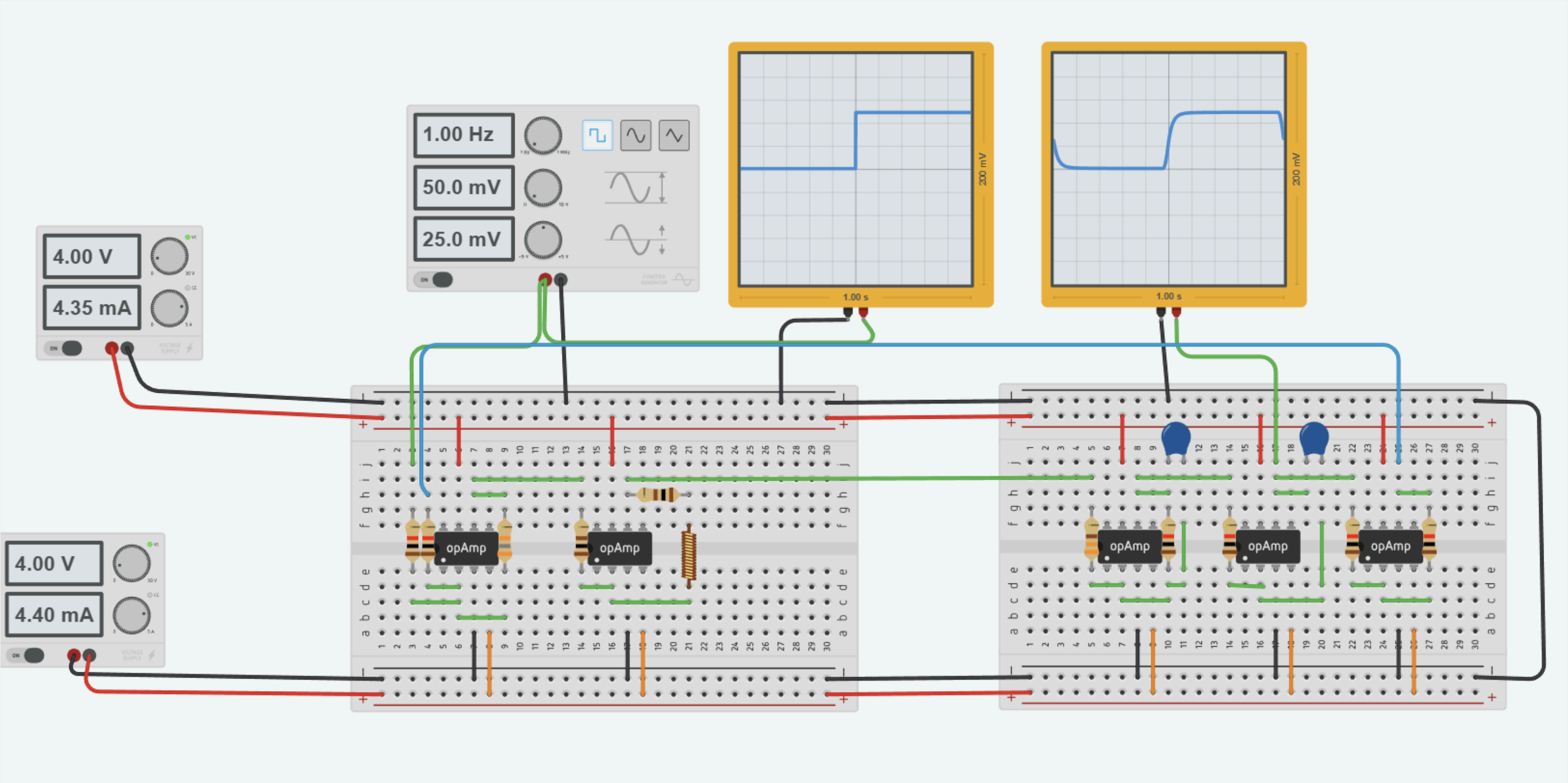
Tinkercad



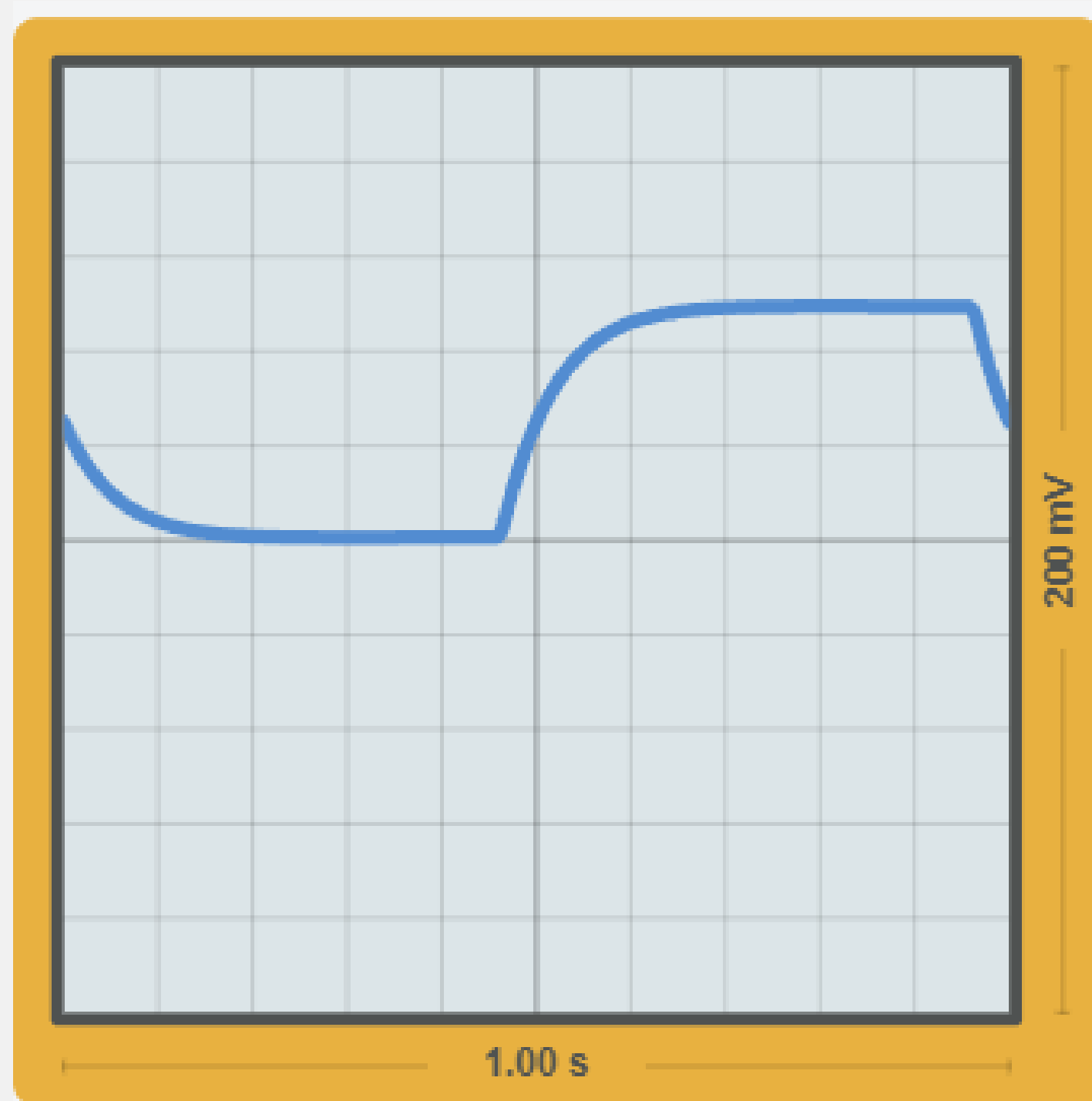
In this section, the pd control and pid control is reproduced in tinkercad. In the field of pd control, as it mentioned in previous part, the K_p and K_d we adopt are 38 and 2.28, the waveforms will be presented in oscilloscope, besides, during the time of research, we have tryed different K_p and k_d value, which are also included and compared with the original $K_p(7)$ and $K_d(0.063)$. In pid control, the values are $K_p=30$, $K_d=2.2$ and $K_i=4$

Yuanhang.Yang

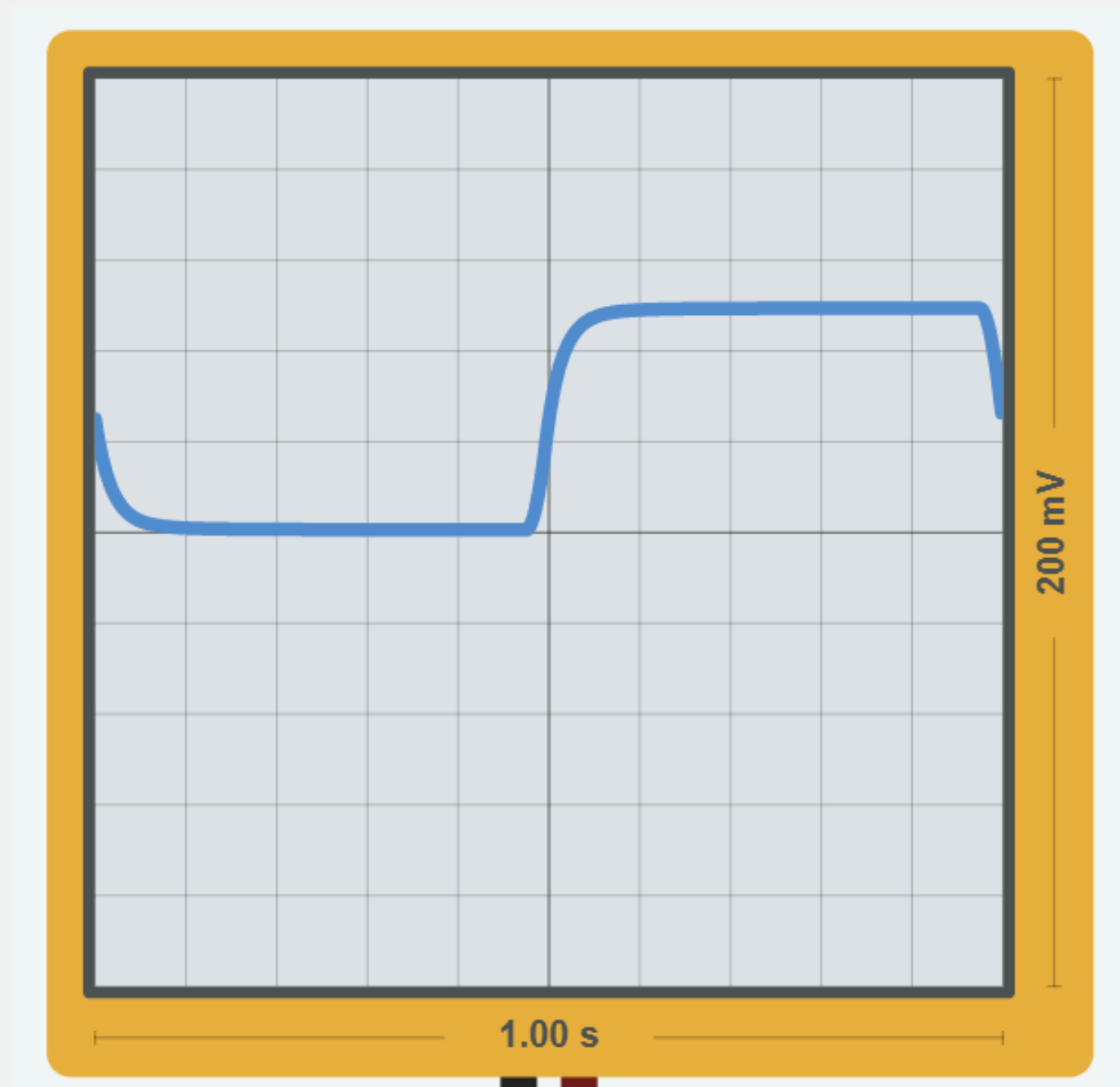
Tinkercad simulation with $K_p=38$ $K_d=2.28$



Yuanhang.Yang

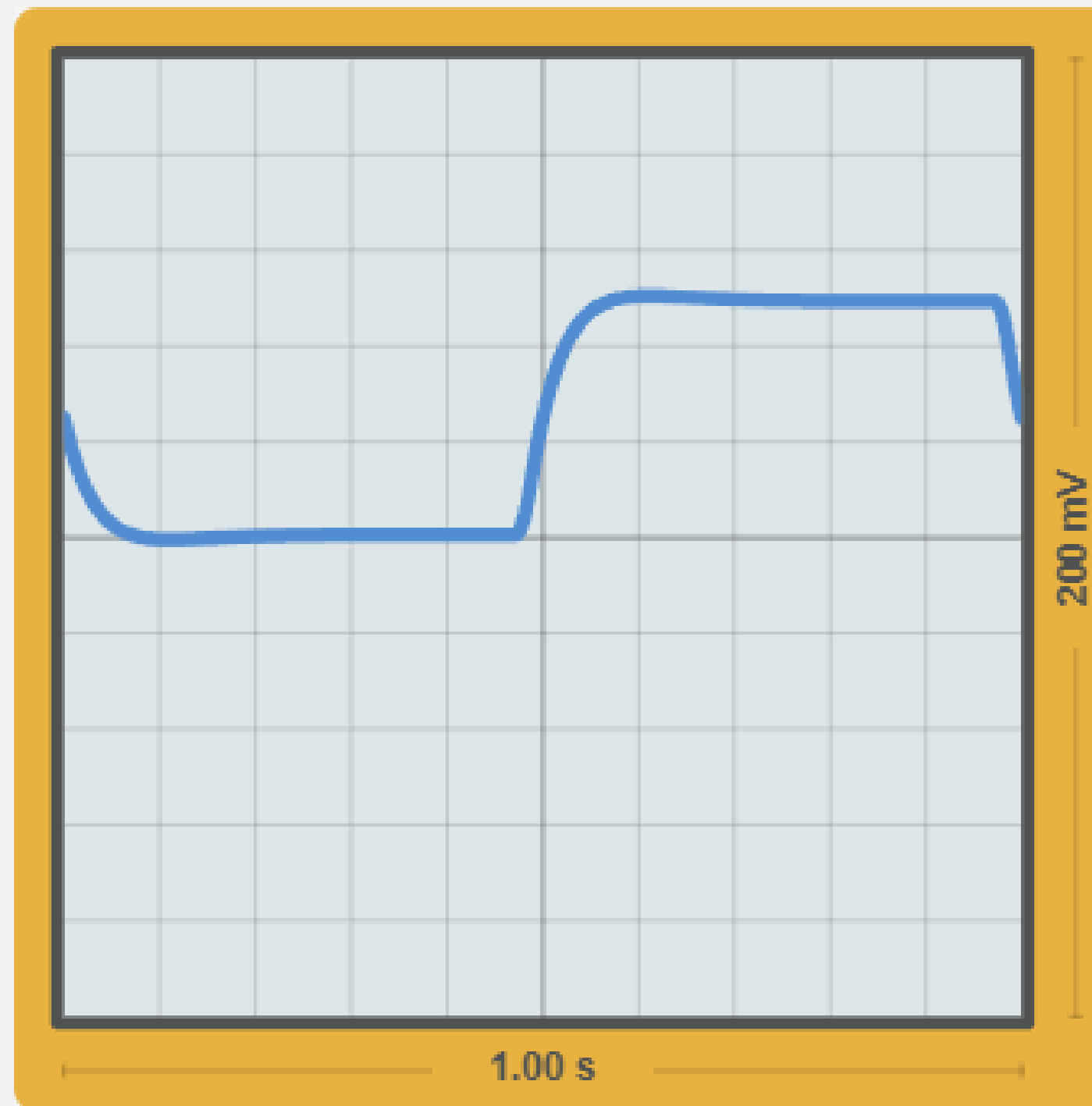


$K_p=7, K_d=0.063$

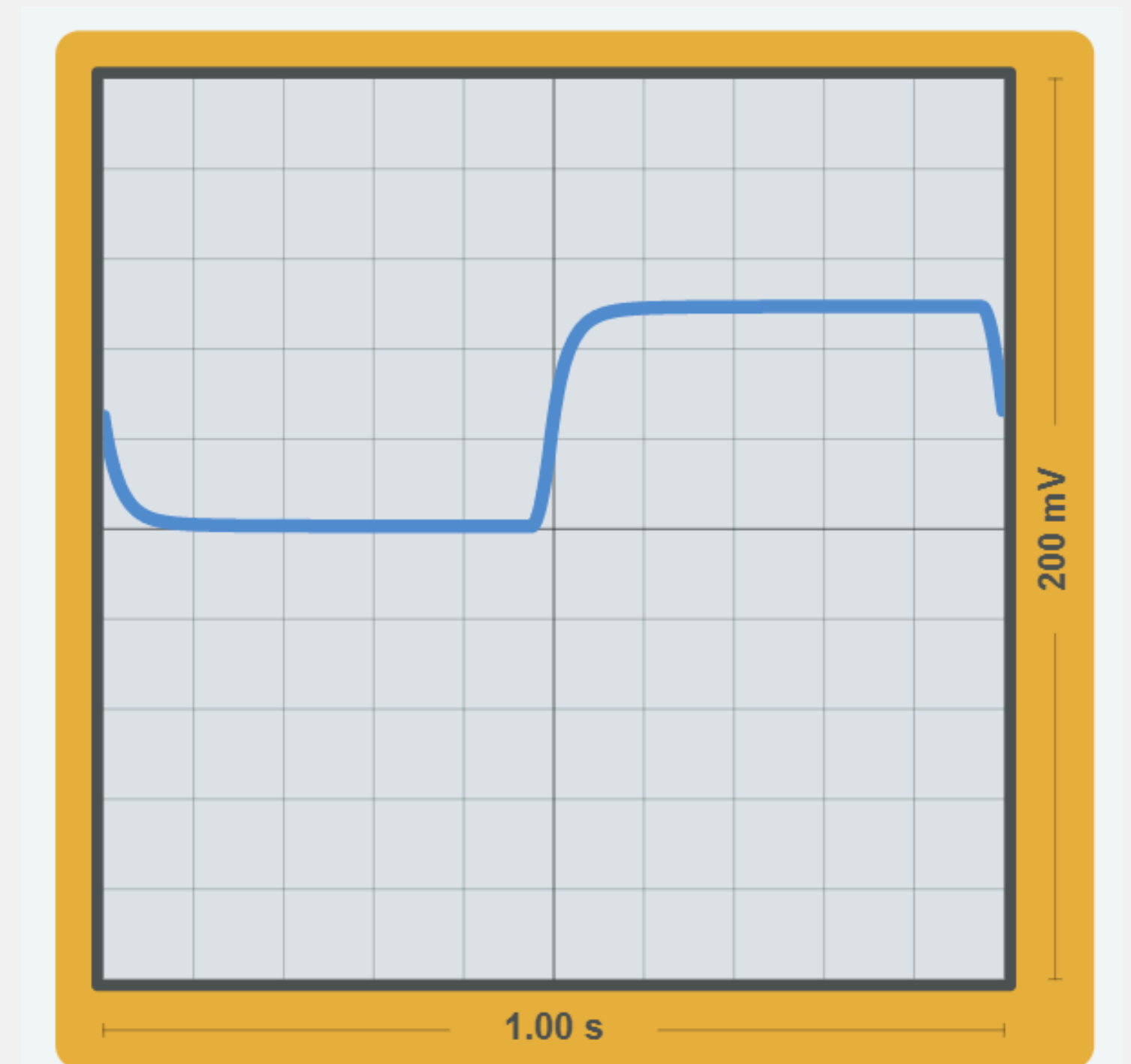


$K_p=38, K_d=2.28$

Yuanhang.Yang



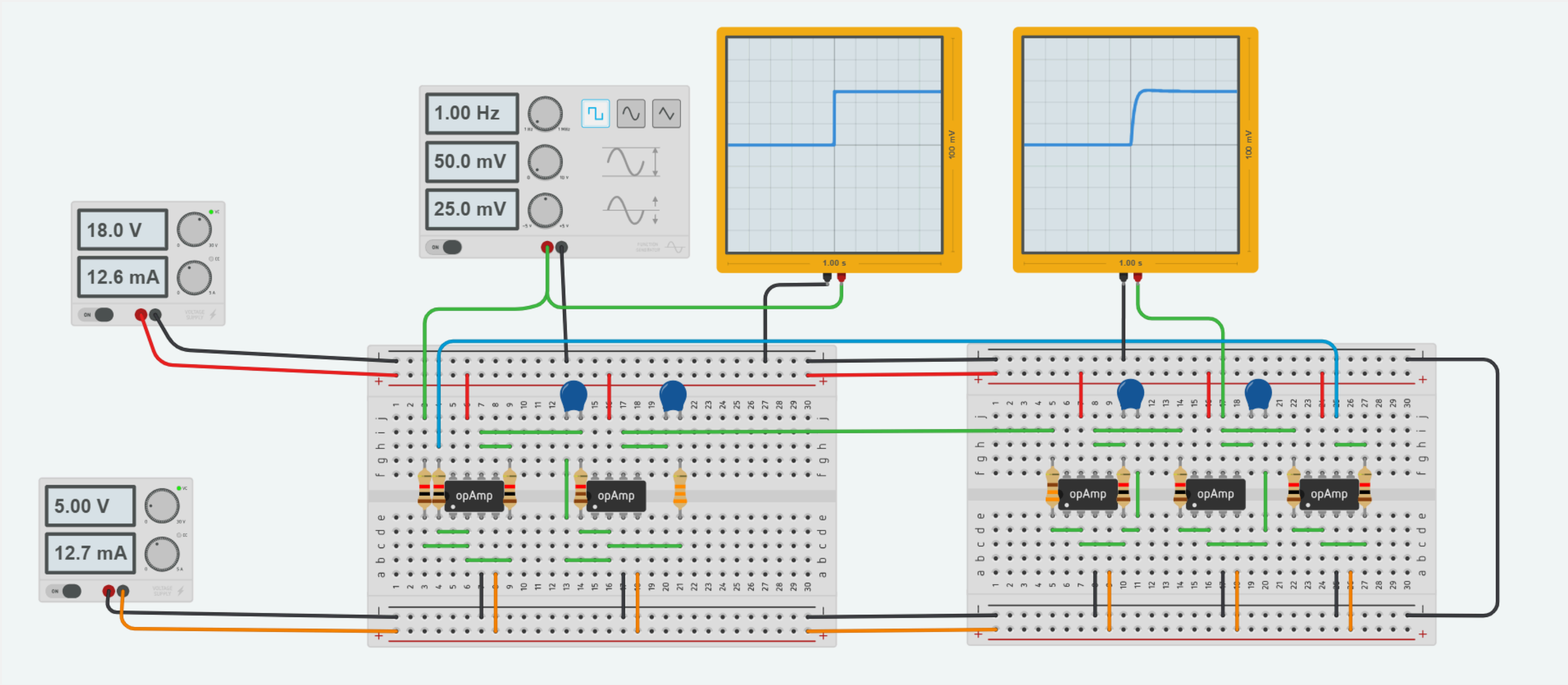
$K_p=21, K_d=1.26$



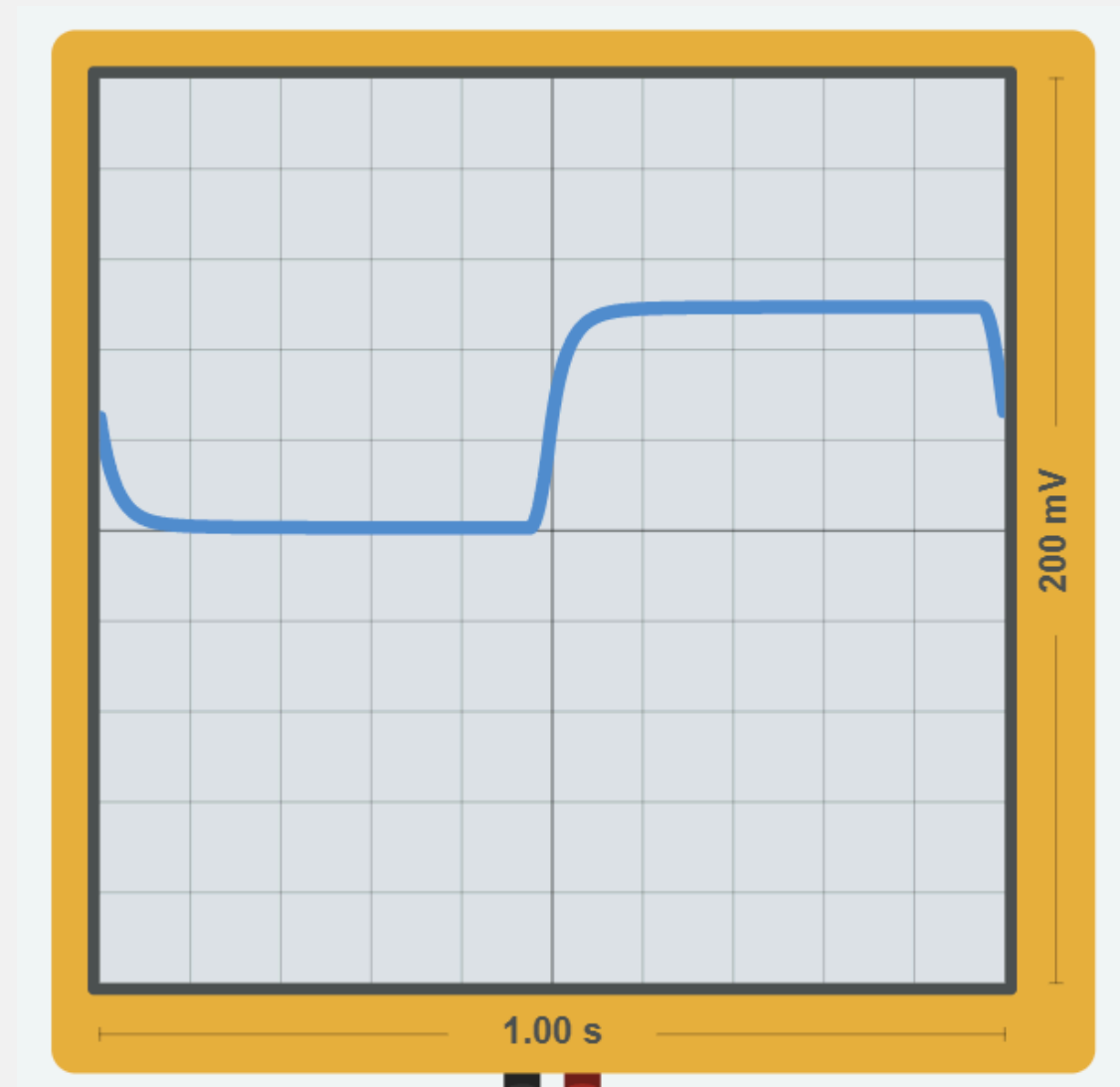
$K_p=38, K_d=2.28$

Yuanhang Yang

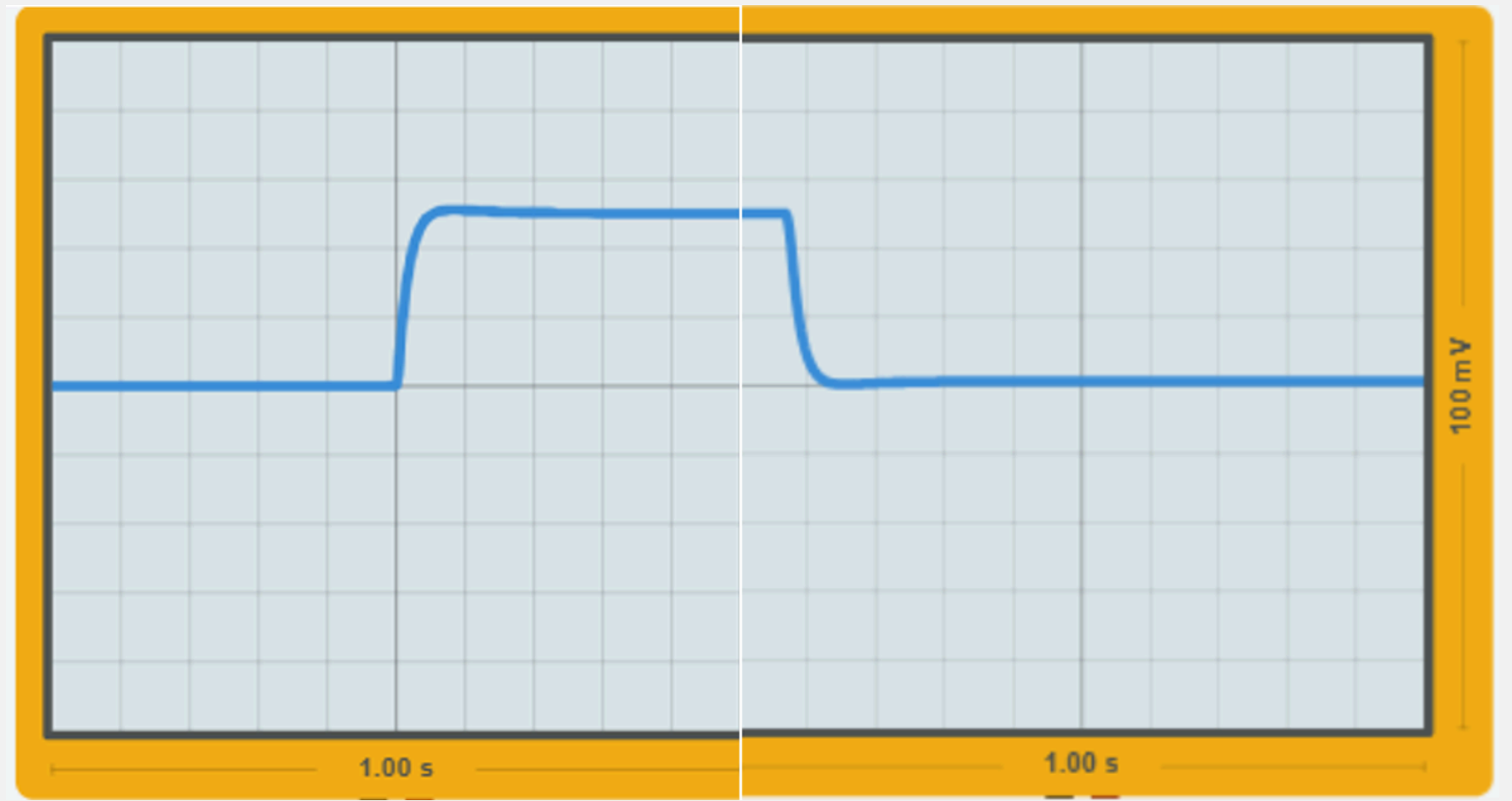
Tinkercad simulation with $K_p=30$, $K_d=2.2$, $K_i=4$



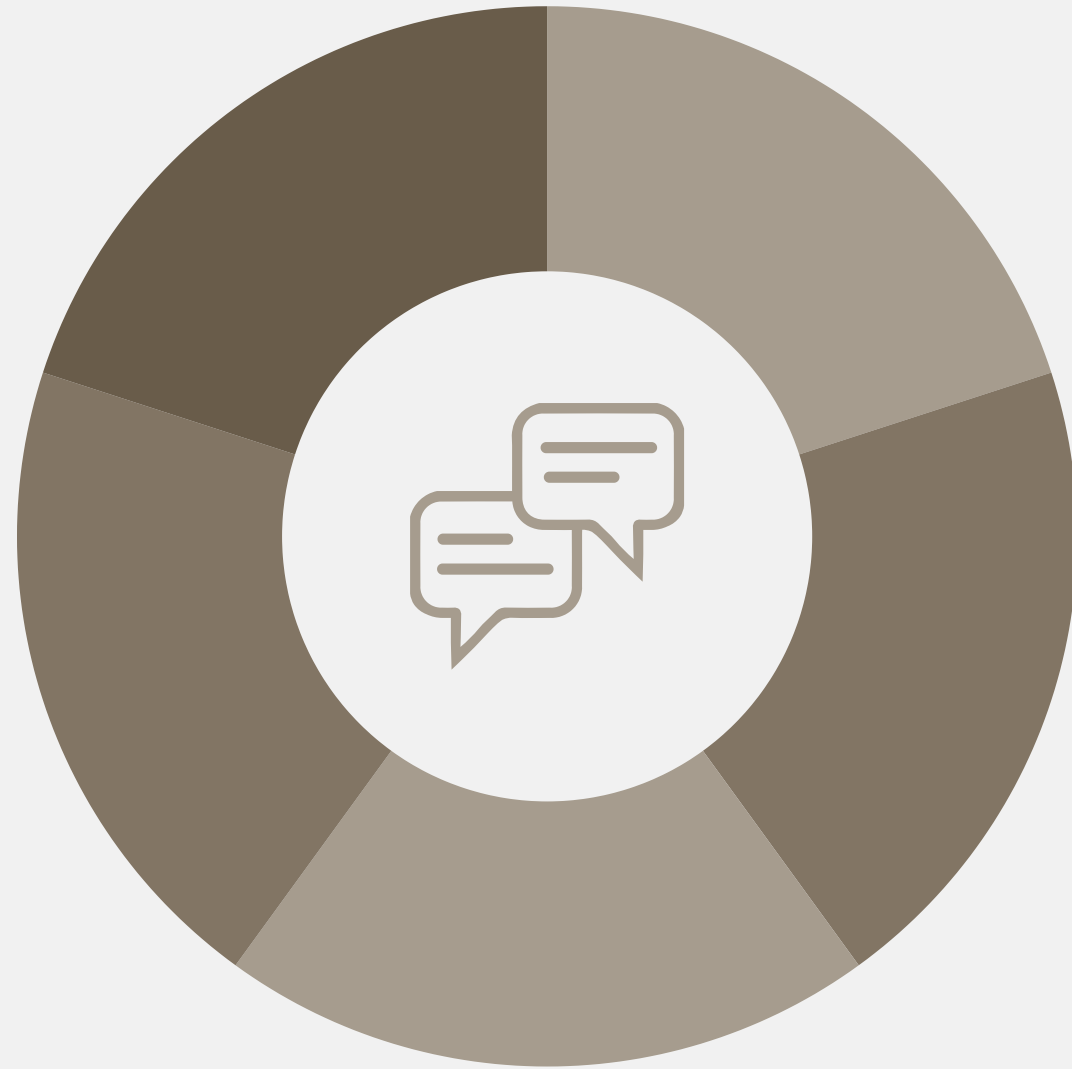
Haokun Shi



PD Control $K_p=38$, $K_d=2.28$



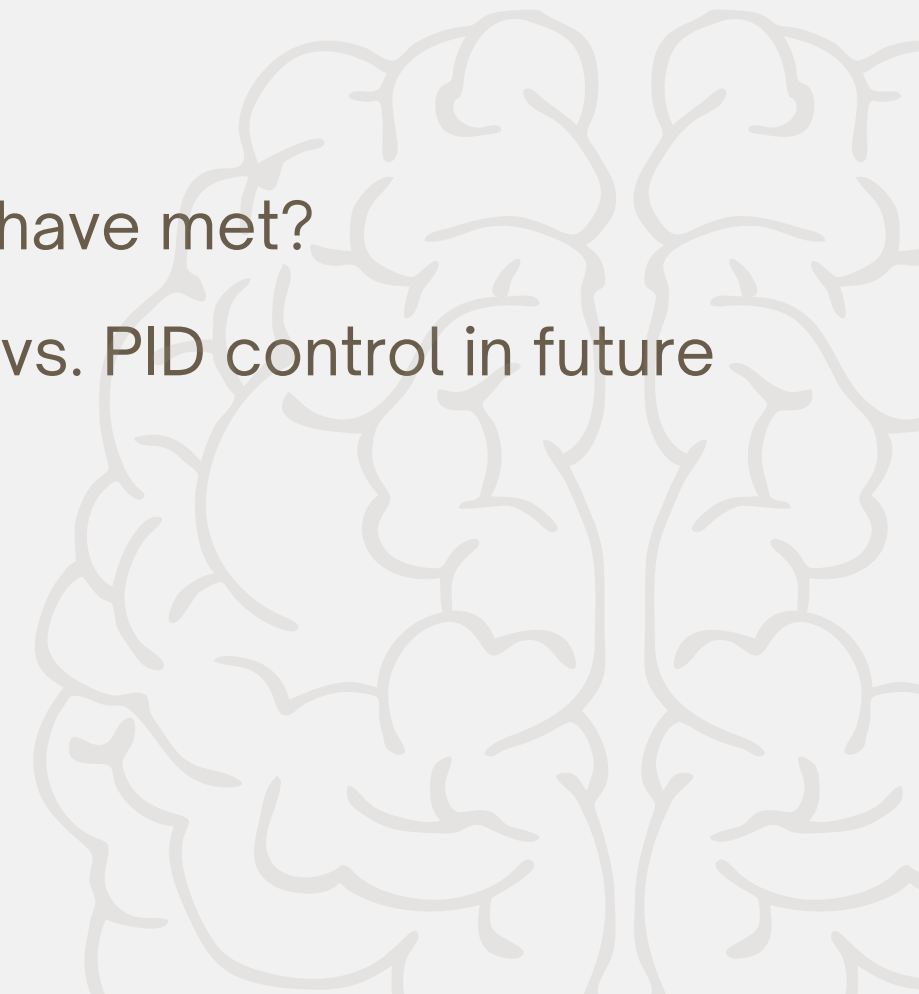
PID Control $K_p=30$, $K_d=2.2$, $K_i=4$



Results & Discussion

DISCUSSION

- What are the challenges we have met?
- what are perspectives of PD vs. PID control in future research?

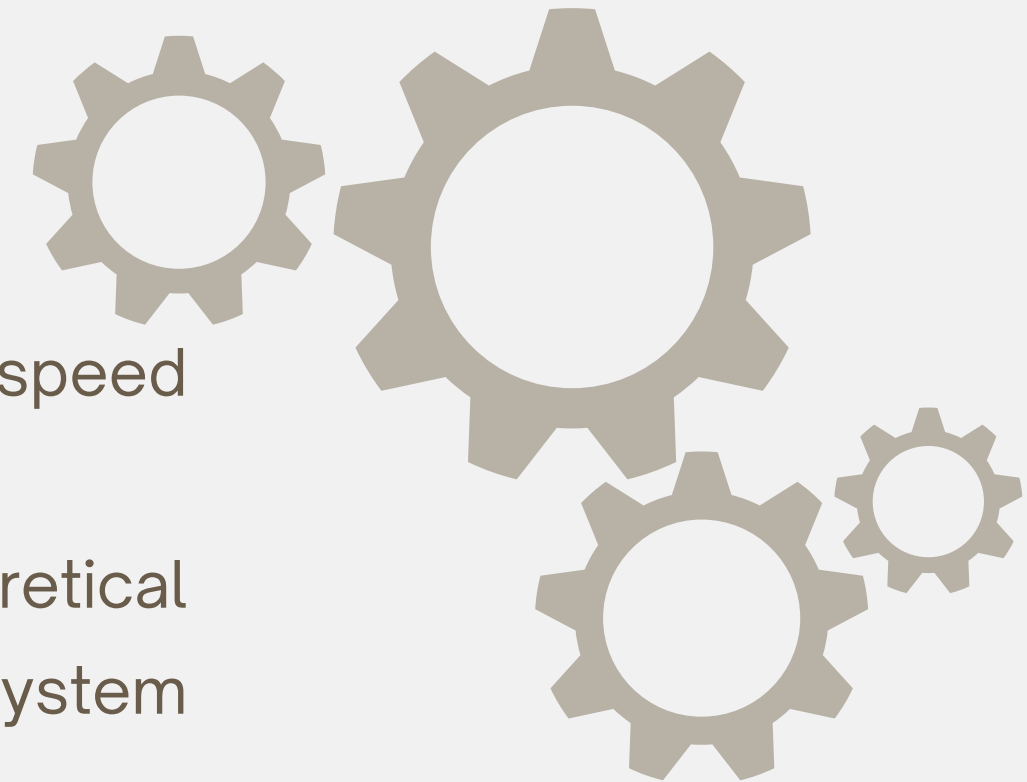


Complexity in Parameter Tuning: Achieving optimal performance with PID controllers requires precise tuning of parameters.

Precision : The distinctive performance of Octave and Falstad

Trade-off Between Speed and Stability – Increasing the gain improved response speed but also increased the risk of instability.

Computational vs. Practical Implementation – While simulations provided theoretical insights, real-world implementation may require additional tuning due to system uncertainties.



Analysis for the waveform

As can be seen from the image, when K_p is 38 and K_d is 2.28, the PD control has a smaller rising time, smaller overshoot, shorter settling time, and significantly improved stability compared to the initial PD control.

Problem encountered

During the simulation process in Tinkercad, I encountered an issue where the operational amplifier's +Vcc and -Vcc supply voltages being too high caused the amplifier to saturate. This issue was resolved by reducing the absolute value of the power supply to 4V.

Yuanhang Yang

Conclusion

PID/PD Control aimed a balance:

stability

fast response

avoid excessive overshoot

Future work could involve adaptive tuning methods or machine learning-based optimization to refine parameter selection dynamically.



Yutong Liu

Reference

1. Circuit Digest. (2018, January 7). What is a PID controller? Working, structure & applications. Retrieved from <https://circuitdigest.com/article/what-is-pid-controller-working-structure-applications>
2. eCircuit Center. (n.d.). Op-amp PID controller. Retrieved from https://www.ecircuitcenter.com/Circuits/op_pid/op_pid.htm
3. ResearchGate. (n.d.). File post: PID control theory and applications. Retrieved from <https://www.researchgate.net/file.PostFileLoader.html?assetKey=AS%3A273635200176128%401442251123954&id=54685991d11b8bc9668b461a>
4. Spezia, R. (2015, November 30). Lesson 17: PID control theory. Southern Illinois University. Retrieved from <https://www.engr.siu.edu/staff/spezia/Web438A/Lecture%20Notes/lesson17et438a.pdf>

Coding Reference

We use coding to simulate and use video making tools to approach a gif animation.

<https://forum.arduino.cc/t/pid-library/333329>

```
1 disp("Hello World");
2 pkg load control; % Load the control system toolbox
3 pkg load signal; % Load the signal processing toolbox
4
5 % Define the plant model (controlled system)
6 s = tf('s');
7 plant = 1 / (s^2 + 2*s + 1); % Second-order system G(s) = 1 / (s^2 + 2s + 1)
8
9 % Define the range of PID parameters
10 Kp_values = linspace(0.5, 5, 50); % Kp varies from 0.5 to 5
11 Kd_values = linspace(0, 2, 50); % Kd varies from 0 to 2
12 Ki_values = linspace(0, 2, 50); % Ki varies from 0 to 2
13
14 % Define step input
15 t = 0:0.1:10;
16 u = ones(size(t)); % Reference input (step function)
17
18 % Create animation
19 figure;
20 for i = 1:length(Kp_values)
21 % Get current PID parameters
22 Kp = Kp_values(i);
23 Kd = Kd_values(i);
24 Ki = Ki_values(i);
25
26 % Define the PID controller
27 C = Kp + Ki/s + Kd*s;
28 closed_loop = feedback(C * plant, 1); % Closed-loop system
29
30 % Compute step response
31 [y, t_out] = step(closed_loop, t);
32
33 % Plot the response
34 clf; hold on;
35 plot(t_out, y, 'b', 'LineWidth', 2);
36 plot(t, u, 'r--', 'LineWidth', 1.5); % Reference input
37 title(sprintf('PID Response: Kp=%.2f, Kd=%.2f, Ki=%.2f', Kp, Kd, Ki));
38 xlabel('Time (s)');
39 ylabel('Response');
40 legend('System Response', 'Reference Input');
41 axis([0 10 0 2]); % Set axis limits
42 grid on;
43
44 pause(0.1); % Pause to create animation effect
45 end
46 Improve the above codes by make a gif of variation
```

The background features a light gray color with abstract geometric elements. In the top right, there are several concentric circles and a vertical line. In the bottom left, there are concentric semi-circles and a vertical line. A horizontal line spans the width of the page, intersected by a vertical line on the left side.

Thank you

22 MARCH 2025